University of Victoria

UNIVERSITY OF VICTORIA

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE 503: Optimization for Machine Learning

Project Report:
Sentiment Analysis of Reviews Using PCA and SVM

Student:
Ming Chen

Student Number:
V01097848

Lab Section:
B01

Your TA:
Ruilin Wang

April 21, 2026

Contents

1 Introduction **4**

2 Optimization **6**

2.1 Support Vector Machine (SVM) 6

2.2 Principal Component Analysis (PCA) 7

3 Solution Methods **9**

3.1 Data Preprocessing 9

3.2 TF-IDF Feature Extraction 9

3.3 PCA Implementation 10

3.4 SVM Classification and Cross-Validation 10

3.5 Experiments Design 11

4 Simulations and Numerical Results **12**

4.1 PCA Variance Analysis & Cross-Validation 12

4.2 Classification Results 13

5 Conclusion **15**

A Python Code **17**

A.1 pca.py 17

A.2 svm_classifier.py 20

A.3 main.py 22

Chapter 1

Introduction

With the rapid growth of e-commerce and social media platforms, consumers routinely share personal opinions and product feedback through online reviews. Platforms such as Amazon, IMDB, and Yelp collectively accumulate millions of user-generated reviews every day. While this wealth of information is valuable, it presents a practical challenge on product manager cannot manually read and assess every individual review. An automated and accurate sentiment analysis system that can classify each review as positive or negative which can offers a significant improvement in operational efficiency.

The core challenge in sentiment analysis lies in transforming unstructured text into numerical representation suitable for machine learning. This project applies principled machine learning pipeline to UCI Sentiment Labelled Sentences dataset[3], which dataset contains 3,000 sentences collected from three distinct platforms. Each platform contributes sentences balanced equally between positive and negative labels, make a well-structured binary classification benchmark that spans three different review domains.

The pipeline addresses two interconnected questions. First, TF-IDF vectorization is used to extract features from the raw text. Second, Principal Component Analysis is explored as a dimensionality reduction step, and its effect on classification performance is evaluated. A key question being whether PCA is effective for sparse high-dimensional text. Finally, a linear Support Vector Machine is employed as the classifier, given its strong theoretical grounding for binary classification via maximum-margin optimization and its well-established performance on text data.

The remainder of this report is organized as follows. Chapter 2 formulates both SVM and PCA as optimization problems. Chapter 3 describes the solution pipeline including data preprocessing, feature extraction, PCA implementation, and cross-validation. Chapter 4 presents experimental results across configurations.

Chapter 2

Optimization

This project addresses two interconnected optimization problems: Principal Component Analysis (PCA) for dimensionality reduction of TF-IDF feature vectors, and Support Vector Machine (SVM) for binary sentiment classification. Both problems are formulated as constrained or regularized optimization problems, where the objective function balances model fitting against complexity control.

2.1 Support Vector Machine (SVM)

Given a training set of P labeled review samples $(x_p, y_p), \quad p = 1, 2, \ldots, P$ where x_p is the TF-IDF feature vector of the p-th review. For the SVM optimization purpose, these are mapped to the sentimental label where

$$y_p = \begin{cases} 1 & \text{if review is positive} \\ -1 & \text{others} \end{cases}$$

the SVM seeks a hyperplane that separates the two classes with maximum margin. The decision function is defined as $f(x) = w^T x + b$ where w is the weight vector and b is the bias. A new review is classified as positive if $f(x) > 0$ and negative otherwise.
The SVM optimization problem in its unconstrained primal form is:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \cdot \frac{1}{P} \sum_{p=1}^{P} \max\big(0,\ 1 - y_p(w^T x_p + b)\big)$$

where the first term is $\frac{1}{2}\|w\|^2$ is the L2 regularization that controls model complexity. The second term is the average hinge loss over all training samples. The factor $1/P$ ensures that the optimal hyperparameter C is invariant to the dataset size.
The hinge loss for single sample is defined as:

$$\xi(y_p, f(x_p)) = \max\big(0,\ 1 - y_p \cdot f(x_p)\big)$$

This loss equals zero when the sample is correctly classified with sufficient margin $y_p \cdot f(x_p) \geq 1$, and increases linearly otherwise. As discussed by Cortes and Vapnik [1], this formulation encourages a sparse solution where only the samples near the decision boundary influence the classifier.
The regularization parameter $C > 0$ controls the trade-off between model complexity and classification error:

> **Large C**: the optimizer prioritizes minimizing the hinge loss (fitting the training data), which may lead to overfitting.
>
> **Small C**: the optimizer prioritizes a simpler model (smaller $\|w\|$), which may lead to underfitting.

This trade-off is directly analogous to the regularization parameter μ used in the **SRMCC** softmax regression, where μ controls the balance between the softmax cost function and the L_2 penalty. In both formulations, the regularized cost function takes the general form:

$$E(w) = Loss(w) + \mu\|w\|^2$$

In the SVM, the parameter C effectively plays the role of $1/\mu$. Thus, a larger parameter C corresponds to weaker regularization, and vice versa. The optimal parameter must be selected empirically through cross-validation, as described in next Chapter.

2.2 Principal Component Analysis (PCA)

The TF-IDF feature vectors may be reduced to lower dimension K using Principal Component Analysis. PCA is formulated as the following constrained optimization problem. Given a mean-centered data matrix $[x_1, x_2, \ldots, x_p]$

where each $x_p \in R^D$, the goal is find a "code book" $C = [c_1, c_2, \ldots, c_k$ with $K < D$ such that each sample can be approximated by:

$$x_p \approx \sum_{k=1}^{K} v_{p,k} c_k = C\, v_p \quad \text{where} \quad v_p \approx C^T x_p$$

the K-dimensional is the encoded representation. Then, the best code book C is found by minimizing the reconstruction error:

$$\underset{\mathbf{C}}{\text{minimize}} \quad f(\mathbf{C}) = \frac{1}{P} \sum_{p=1}^{P} \|\mathbf{C}\mathbf{C}^T \mathbf{x}_p - \mathbf{x}_p\|_2^2$$

$$\text{subject to:} \quad C^T C = I_K (\text{columns of } C \text{ orthonormal})$$

minimizing $f(C)$ is equivalent to maximizing the sum of the first K eigenvalues.

In this project , the PCA algorithm was implemented from scratch using NumPy rather than the build-in PCA module in scikit-learn (sklearn), to demonstrate understanding of the mathematical procedure. The handcrafted implementation was validated against the sklearn version, achieving numerical agreement within floating-point precision. The PCA serves a role analogous to the HOG feature transformation used. Both Transform the original feature representation into a more compact form before classification.

However, as following the section will show, the effectiveness of PCA on text TF-IDF features differs significantly from HOG on image features, due to fundamental differences in the eigenvalue spectra.

Chapter 3

Solution Methods

The solution pipeline has six stages implemented as separate Python modules: data cleaning and combining, train and test datasets splitting, TF-IDF feature extraction, PCA dimensionality reduction, SVM classification with cross-validation, and result visualization.

3.1 Data Preprocessing

The dataset is the Sentiment Labeled Sentences Data Set from UCI[3], total containing 3,000 sentences from three platforms: Amazon, IMDB, and Yelp. Each platform has 1,000 sentences and each sentence is positive or negative, with 500 samples per class per platform. The three files are loaded and combined by the *text_combined()* function, which cleans each file by converting text to lowercase, removing non-alphabetic characters, and compressing whitespace. After cleaning, all reviews then split into 80% training and 20% test using stratified sampling.

3.2 TF-IDF Feature Extraction

Text is converted to numerical vectors using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization:

$$\textit{TF-IDF}\ (t, d) = TF(t, d) \times IDF(t)$$

where $TF(t,d) = 1 + log(f_{t,d})$ uses sublinear scaling to reduce the dominance of high-frequency words, and $IDF(t) = log((1+P)/(1+df(t))) + 1$ is the smoothed inverse document frequency [4]. Each document vector is L2-normalized. The vectorizer uses max 2,300 features with *unigram-bigram* range. Bigrams are essential for sentiment analysis because phrases like "not good" carry opposite meaning to "good" alone. The resulting feature matrix has 2,126 dimension with 99.77% sparsity.

3.3 PCA Implementation

PCA is implemented from scratch using only NumPy. First, each feature is standardized to zero mean and unit variance, and the data is centered. Secondly, the covariance matrix $S = (1/(N-1))X_c^T X_c$ is computed and its symmetry is verified. After verification is complete, the eigenvalues and eigenvectors of symmetric matrix S are computed via NumPy, and chosen the top K eigenvectors as the "code book". Finally, the data is projected via $v_p = C^T x_p$. The implementation was validated against sklearn with eigenvalues matching within 0.5%.

3.4 SVM Classification and Cross-Validation

To handle this project, I employ the primal linear SVM solver provided by LIBLINEAR[2], which implements an optimization framework similar to the one proposed by Weston and Watkins[5] for solving multi-class pattern recognition in a single step. The regularization parameter C is selected by 5-fold stratified cross-validation on the training data only. For each candidate $C \in \{0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0\}$, the training set is partitioned into 5 folds with preserved class balance, and the mean validation accuracy is recorded. This mirrors of the RMSE validation curve was used to find the optimal μ.

Table 3.1: Summary of the three experimental configurations.

Exp.	**Method**	**Features**	**Purpose**
1	Linear SVM + TF-IDF	2,097D	Baseline (no PCA)
2	Linear SVM + PCA-450D	450D	Aggressive reduction
3	Linear SVM + PCA-600D	600D	Moderate reduction

3.5 Experiments Design

Three experiments are conducted with the best C value identified by cross-validation. As above table 3.1 shows: experiment 1 vs. 2 and 3 evaluates the effect of PCA dimensionality reduction at different compression levels. Experiment 2 and 3 reveals the trade-off between compression ratio and classification accuracy. All experiments use the same training and testing datasets, and the same C value for fair comparison.

Chapter 4

Simulations and Numerical Results

This section presents the experimental results and provides analysis for each of generated figures. All experiments use the best regularization parameter $C = 0.5$, which was identified by 5-fold cross-validation.

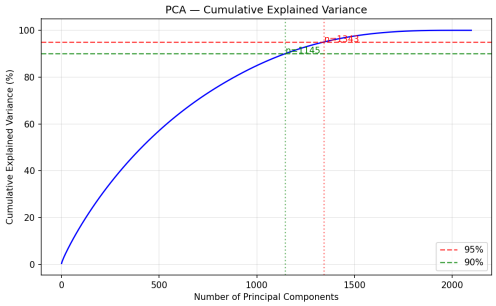
4.1 PCA Variance Analysis & Cross-Validation

The following figures show the cumulative explained variance number of principal components 4.1a and the cross-validation accuracy versus C 4.1b.

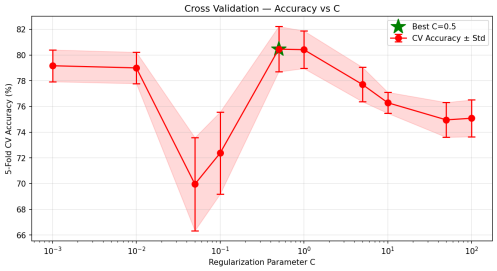
To retain 90% of the variance, 1,145 components are required out of 2,097 total dimensions. Consequently, **PCA-450D** retains only approximately 50% of the total variance, and **PCA-600D** retains approximately above 60%. Notably, PCA-600D achieves slightly lower accuracy than PCA-450D despite retaining more variance. This counterintuitive result is likely attributable to the additional principal components introducing noise dimensions — directions of low but non-zero variance that carry no sentiment signal — which slightly mislead the linear classifier. It suggests that beyond a certain threshold, adding more components from a flat, diffuse eigenvalue spectrum does not improve and may marginally hurt classification.

The curve exhibits the same U-shaped trade-off. The SVM model under-fits

when C is less than 0.1 and once C values are greater than 5, the model overfits the training data. The optimal value $C = 0.5$ achieves a CV accuracy of 80.46% $\pm$ 1.76%. This value reflects a moderate regularization strength because it is large enough to allow the model to fit the training data well, yet small enough to prevent overfitting to noise in the high-dimensional TF-IDF feature space.



(a) Cumulative explained variance versus the number of principal components.



(b) 5-fold cross-validation accuracy versus regularization parameter C. The green star marks the optimal $C = 0.5$.

Figure 4.1: PCA Variance and Cross-Validation Results

4.2 Classification Results

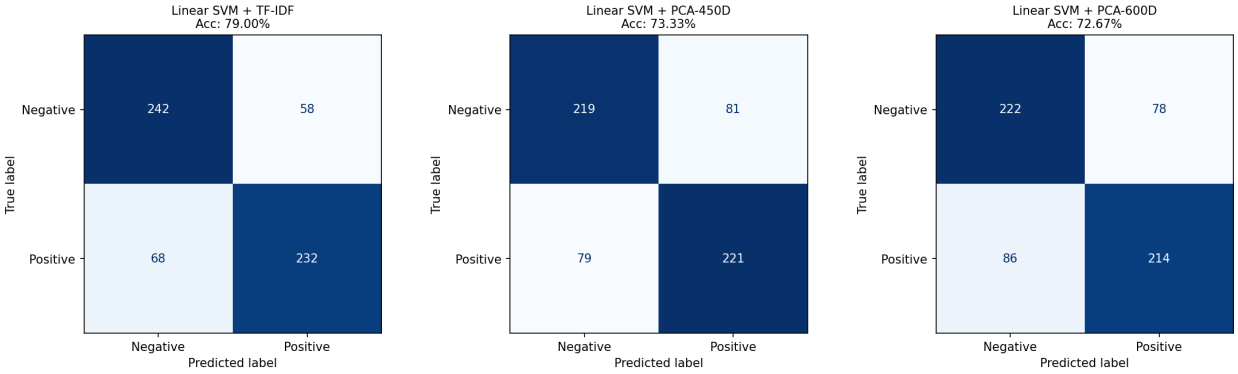
Table 4.1 and Figure 4.2a summarize the three experiments. The Accuracy, F1, and Recall are nearly identical in each row because the dataset is perfectly balanced. Metrics would diverge when the databases are imbalanced.

The key finding is that PCA consistently degrades accuracy on TF-IDF features. Reducing from 2,097D to 450D loses 5.67% points, and 600D loses 6.33% points. The efficacy of PCA is highly dependent on the underlying data distribution. In image processing, information is often concentrated in a few dominant eigenvectors, PCA or HOG dimensionality reduction can enhance classification accuracy by filtering noise.

In contrast, for TF-IDF text features, sentiment is widely distributed across many terms, and aggressive dimensionality reduction often results in the loss of vital information. Furthermore, while TF-IDF benefits from the computational efficiency of sparse matrices, PCA forces these into a dense format, substantially increasing training duration.

Table 4.1: Experimental results with best $C = 0.5$.

Method	**Accuracy**	**F1**	**Recall**	**Train (s)**	**Test (s)**
Linear SVM + TF-IDF	79.00%	79.0%	79.0%	0.007	0.0002
Linear SVM + PCA-450D	73.33%	73.3%	73.3%	4.244	0.0008
Linear SVM + PCA-600D	72.67%	72.7%	72.7%	6.707	0.0003



(a) Confusion matrices for the three configurations.

Chapter 5

Conclusion

This project built a sentiment analysis pipeline using TF-IDF feature extraction, PCA dimensionality reduction, and linear SVM classification on the UCI Sentiment Labelled Sentences dataset. The baseline linear SVM on 2,097-dimensional TF-IDF features achieved 79.00% test accuracy with $C = 0.5$ selected via 5-fold cross-validation. Applying PCA to 450D and 600D reduced accuracy by approximately 5.67% and 6.33%, respectively, demonstrating that PCA is less effective for sparse text features where sentiment information is distributed across many dimensions rather than concentrated in a few dominant eigenvectors.

For future work, nonlinear kernels such as RBF may better capture complex boundaries in reduced feature spaces, and sparse-friendly methods such as Truncated SVD could serve as more suitable alternatives to PCA for text data.

Reference

[1] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine learning* 20.3 (1995), pp. 273–297.

[2] Rong-En Fan et al. "LIBLINEAR: A library for large linear classification". In: *Journal of machine learning research* 9 (2008), pp. 1871–1874.

[3] Dimitrios Kotzias. *Sentiment Labelled Sentences*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C57604. 2015.

[4] Gerard Salton and Christopher Buckley. "Term-weighting approaches in automatic text retrieval". In: *Information Processing & Management* 24.5 (1988), pp. 513–523. DOI: 10.1016/0306-4573(88)90021-0.

[5] Jason Weston and Chris Watkins. "Support vector machines for multi-class pattern recognition". In: *The European Symposium on Artificial Neural Networks (ESANN)*. 1999, pp. 219–224. URL: https://www.esann.org/sites/default/files/proceedings/legacy/es1999-461.pdf.

Appendix A

Python Code

A.1 pca.py

```
import numpy as np


class MyPCA:

    optimization:
      max_W  trace(W'*S*W)
      s.t.   W'*W = I

    equivalent
      min_W     abs(x_i - W*W'*x_i)^2
      s.t.   W'*W = I

    def __init__(self, n_components):
        self.n_components = n_components
        self.mean_ = None
        self.components_ = None                  # W_k, shape
            (D, k)
        self.eigenvalues_ = None                 # top k
            eigenvalues
        self.all_eigenvalues_ = None             # all
            eigenvalues (for variance curve)
        self.explained_variance_ratio_ = None    # variance of
            principal components

    def fit(self, X):
        N, D = X.shape

```

```
25          self.mean_ = X.mean(axis=0)  # shape (D,)
26          X_centered = X - self.mean_  # shape (N, D)
27
28          S = (X_centered.T @ X_centered) / (N - 1)
29          eigenvalues, eigenvectors = np.linalg.eigh(S)
30          eigenvalues = eigenvalues[::-1]  # shape (D,)
31          eigenvectors = eigenvectors[:, ::-1]  # shape (D, D)
32          eigenvalues = np.maximum(eigenvalues, 0)        #
               nonzero clip numerical noise (eigenvalues should
               be     0)
33          self.all_eigenvalues_ = eigenvalues.copy()      #
               store all eigenvalues (for variance curve)
34          k = self.n_components
35          self.components_ = eigenvectors[:, :k]  # shape (D, k
               )
36          self.eigenvalues_ = eigenvalues[:k]  # shape (k,)
37          total_variance = eigenvalues.sum()
38          self.explained_variance_ratio_ = eigenvalues[:k] /
               total_variance # variance ratio by principal
               component =  _i  /   _j
39          return self
40
41      def transform(self, X):
42          X_centered = X - self.mean_
43          X_reduced = X_centered @ self.components_
44          return X_reduced
45
46      def fit_transform(self, X):
47          self.fit(X)
48          return self.transform(X)
49
50      def inverse_transform(self, X_reduced):
51          return X_reduced @ self.components_.T + self.mean_
52
53      def get_cumulative_variance(self):
54          total_var = self.all_eigenvalues_.sum()
55          return np.cumsum(self.all_eigenvalues_) / total_var *
               100
56
57
58  class MyStandardScaler:
59
60      mathematic equation: x_scaled = (x - mean) / std
61
62      def __init__(self):
63          self.mean_ = None
64          self.std_ = None
65
66      def fit(self, X):
```

```
67          self.mean_ = X.mean(axis=0)
                                                # shape (D,)
68          self.std_ = X.std(axis=0, ddof=0)
                                            # population std
69          self.std_ = np.where(self.std_ < 1e-10, 1.0, self.
                std_)     # prevent division by zero
70          return self
71
72      def transform(self, X):
73          return (X - self.mean_) / self.std_
74
75      def fit_transform(self, X):
76          self.fit(X)
77          return self.transform(X)
78
79
80  def apply_pca(X_train_tfidf, X_test_tfidf, n_components=100):
81
82      Apply handcrafted PCA to TF-IDF features.
83
84      :return: X_train_pca, X_test_pca, scaler, pca
85
86      X_train_dense = X_train_tfidf.toarray()
87      X_test_dense = X_test_tfidf.toarray()
88      n_components = min(n_components, X_train_dense.shape[1] -
            1)
89      scaler = MyStandardScaler()
90      X_train_scaled = scaler.fit_transform(X_train_dense)
91      X_test_scaled = scaler.transform(X_test_dense)
92      pca = MyPCA(n_components=n_components)
93      X_train_pca = pca.fit_transform(X_train_scaled)
94      X_test_pca = pca.transform(X_test_scaled)
95
96      explained = sum(pca.explained_variance_ratio_) * 100
97      print(f \n[MyPCA] {X_train_dense.shape[1]}D     {
            n_components}D )
98      print(f   cumulative variance: {explained:.2f}% )
99      return X_train_pca, X_test_pca, scaler, pca
100
101
102 def optimal_dim(X_train_tfidf, threshold=85):
103
104     Find optimal number of principal components using
            handcrafted PCA.
105     :return: n_optimal, cumulative_var
106
107     X_dense = X_train_tfidf.toarray()
108     scaler = MyStandardScaler()
109     X_scaled = scaler.fit_transform(X_dense)
```

```
110
111     max_comp = min(X_scaled.shape[0], X_scaled.shape[1])
112     pca_full = MyPCA(n_components=max_comp)
113     pca_full.fit(X_scaled)
114     cumulative_var = pca_full.get_cumulative_variance()
115     n_optimal = int(np.argmax(cumulative_var >= threshold) +
            1)
116     return n_optimal, cumulative_var
```

A.2 svm_classifier.py

```
1  import numpy as np
2  import time
3  from sklearn.svm import LinearSVC
4  from sklearn.metrics import (
5      accuracy_score, confusion_matrix, f1_score, recall_score
6  )
7  from sklearn.model_selection import cross_val_score,
       StratifiedKFold
8
9
10 def run_svm(X_train, X_test, y_train, y_test,
11               name= SVM , C=1.0, seed=123):
12
13     Run a single SVM experiment
14     :return: dict with all results
15
16     model = LinearSVC(C=C, max_iter=10000, random_state=seed,
           loss='hinge')
17
18     t0 = time.time()
19     model.fit(X_train, y_train)
20     train_time = time.time() - t0
21
22     t0 = time.time()
23     y_pred = model.predict(X_test)
24     test_time = time.time() - t0
25
26     acc = accuracy_score(y_test, y_pred)
27     f1 = f1_score(y_test, y_pred, average='weighted')
28     recall = recall_score(y_test, y_pred, average='weighted')
29     cm = confusion_matrix(y_test, y_pred)
30     return {
31         'name': name, 'C': C,
32         'accuracy': acc, 'f1': f1, 'recall': recall, 'cm': cm
               ,
```

```
        'train_time': train_time, 'test_time': test_time,
        'y_pred': y_pred, 'model': model
    }


def tune_C_with_cv(X_train, y_train, cv_folds=5, C_values=None):

    Tune regularization parameter C using K-Fold Cross Validation.
    Only uses training data     test set is untouched.

    :return: C_values, cv_mean, cv_std, best_C

    if C_values is None:
        C_values = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]

    cv_mean = []
    cv_std = []

    cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=42)

    for C in C_values:
        model = LinearSVC(C=C, max_iter=10000, random_state=42, loss='hinge')
        scores = cross_val_score(model, X_train, y_train,
                                 cv=cv, scoring='accuracy', n_jobs=-1)
        cv_mean.append(scores.mean())
        cv_std.append(scores.std())

    best_idx = int(np.argmax(cv_mean))
    best_C = C_values[best_idx]
    return C_values, cv_mean, cv_std, best_C


def run_all_experiments(X_train_tfidf, X_test_tfidf, pca_results,
                        y_train, y_test, best_C=1.0):

    Run comparison experiments.
    :return: list of result dicts

    results = []

    results.append(run_svm(
        X_train_tfidf, X_test_tfidf, y_train, y_test,
```

```
        name= Linear SVM + TF-IDF , C=best_C
    ))

    for dim in sorted(pca_results.keys()):
        results.append(run_svm(
            pca_results[dim]['X_train'], pca_results[dim]['
                X_test'],
            y_train, y_test,
            name=f Linear SVM + PCA-{dim}D , C=best_C
        ))
    return results
```

A.3 main.py

```
import re

from data_cleaner import text_combined, split_data
from feature_engineering import tfidf_features, inspect_tfidf
from pca import apply_pca, optimal_dim
from svm_classifier import tune_C_with_cv,
    run_all_experiments
from visualization import *

CONFIG = {
    # data
    'data_folder':      data/raw ,
    'data_files': [ amazon_cells_labelled ,  imdb_labelled ,
        yelp_labelled ],
    'test_size':        0.2,
    'random_seed':      123,

    # TF-IDF
    'max_features':     2300,
    'ngram_range':      (1, 2),

    # PCA
    'pca_dims':         [600, 450],    # dimensions to test

    # Cross Validation
    'cv_folds':         5,
    'C_values':         [0.001, 0.01, 0.05, 0.1, 0.5, 1.0,
        5.0, 10.0, 50.0, 100.0]
}
```

```python
def main():
    df = text_combined(CONFIG['data_folder'], CONFIG['
        data_files'])

    train_dataset, test_dataset = split_data(
        df, test_size=CONFIG['test_size'], seed=CONFIG['
            random_seed']
    )
    y_train = train_dataset[1]
    y_test = test_dataset[1]


    X_train_tfidf, X_test_tfidf, vectorizer = tfidf_features(
        train_dataset, test_dataset,
        max=CONFIG['max_features'],
        ngram=CONFIG['ngram_range']
    )
    for i in range(min(3, X_train_tfidf.shape[0])):
        inspect_tfidf(X_train_tfidf, vectorizer, sample_index
            =i)

    n_optimal, cumvar = optimal_dim(X_train_tfidf, threshold
        =85)
    plot_pca_variance(cumvar)

    pca_results = {}
    for n_comp in CONFIG['pca_dims']:
        print(f \n  --- PCA {n_comp}D --- )
        X_tr_pca, X_te_pca, scaler, pca_model = apply_pca(
            X_train_tfidf, X_test_tfidf, n_components=n_comp
        )
        pca_results[n_comp] = {
            'X_train': X_tr_pca,
            'X_test': X_te_pca,
            'scaler': scaler,
            'pca': pca_model
        }

    C_values, cv_mean, cv_std, best_C = tune_C_with_cv(
        X_train_tfidf, y_train,
        cv_folds=CONFIG['cv_folds'],
        C_values=CONFIG['C_values']
    )
    plot_cv_curve(C_values, cv_mean, cv_std)

    all_results = run_all_experiments(
        X_train_tfidf, X_test_tfidf,
        pca_results, y_train, y_test,
        best_C=best_C
```

```
    )

    print_summary(all_results)
    plot_confusion_matrices(all_results)


if __name__ == __main__ :
    main()
```